

HEALTH NORTH

Documentation Technique

Modélisation des données & Script SQL

Projet	Health North — Application de réservation médicale
Type	BTS SIO option B SLAM — Épreuve E6
Stack technique	Django 6 / PostgreSQL / Python 3.12
Hébergement	Render.com + Supabase (PostgreSQL)
URL production	https://healthnorth.onrender.com
Repository	https://github.com/WalterCorp/healthnorth

1. Présentation du projet

Health North est une application web de réservation de rendez-vous médicaux développée dans le cadre du projet E6 du BTS SIO option SLAM. Elle permet aux patients de créer un compte, de prendre un rendez-vous auprès d'un spécialiste dans une clinique, et de déposer des documents médicaux.

L'application est développée avec le framework Django (Python) et s'appuie sur une base de données PostgreSQL. Elle est déployée en production sur Render.com avec une base de données hébergée sur Supabase.

1.1 Applications Django

Le projet est structuré en deux applications Django distinctes :

- accounts — gestion des utilisateurs (inscription, connexion, profil, mot de passe)
- appointments — gestion des rendez-vous, spécialités, cliniques, spécialistes et documents

1.2 Fonctionnalités implémentées

- Mission 1 : Inscription et connexion d'un utilisateur
- Mission 2 : Modification du profil et du mot de passe
- Mission 3 : Parcours complet de prise de rendez-vous (région → ville → clinique → spécialiste → examen → date → documents)
- Mission 4 : Consultation de la liste des rendez-vous du patient
- Mission 5 : Interface d'administration Django (backend)

2. Modèle Conceptuel de Données (MCD)

Le MCD représente les entités métier de l'application et leurs relations, indépendamment de toute technologie. Il a été conçu pour répondre aux besoins fonctionnels de Health North : un patient peut prendre un rendez-vous auprès d'un spécialiste dans une clinique, pour un type d'examen donné, et y déposer des documents.

Note : Le diagramme SVG du MCD est disponible en fichier joint (healthnorth_mcd.svg).

2.1 Entités et leurs attributs

USER (accounts_user)

Représente tout utilisateur de l'application : patient, spécialiste ou administrateur. Le modèle User étend AbstractUser de Django, ce qui évite de recréer from scratch la gestion des mots de passe, des sessions et des permissions.

Attributs ajoutés au-delà d'AbstractUser : phone, address, date_of_birth. Ces champs sont propres au contexte médical de Health North.

SPECIALTY (appointments_specialty)

Représente une spécialité médicale (Cardiologie, Radiologie, Dermatologie, etc.). Elle porte une durée moyenne de rendez-vous (duration_minutes) qui sert de valeur par défaut. C'est une entité de référentiel : stable, peu modifiée, administrée par l'équipe médicale.

EXAM TYPE (appointments_examtype)

Représente un type d'examen médical précis (ECG, IRM, Scanner, etc.) rattaché à une spécialité. La relation avec Specialty est une clé étrangère avec ON DELETE SET NULL : si une spécialité est supprimée, les examens associés restent en base mais se retrouvent sans spécialité, ce qui évite une perte de données.

CLINIC (appointments_clinic)

Représente une clinique ou un laboratoire Health North sur le territoire français. La région est stockée sous forme de slug normalisé (ex : 'ile-de-france') plutôt que de libellé accentué, afin de fiabiliser les filtres dynamiques des API JSON utilisées dans le formulaire de prise de rendez-vous.

SPECIALIST (appointments_specialist)

Représente un médecin spécialiste. C'est une extension du modèle User via une relation OneToOneField : un spécialiste est avant tout un utilisateur avec des droits et un profil médical supplémentaire (spécialité, bio, cliniques). La relation OneToOne garantit qu'un compte utilisateur ne peut être lié qu'à un seul profil spécialiste.

La relation avec Clinic est ManyToMany : un spécialiste peut exercer dans plusieurs cliniques, et une clinique peut accueillir plusieurs spécialistes. Django génère automatiquement la table de liaison appointments_specialist_clinics.

APPOINTMENT (appointments_appointment)

Entité centrale de l'application. Elle matérialise un rendez-vous entre un patient (FK vers User), un spécialiste (FK vers Specialist), dans une clinique (FK nullable vers Clinic), pour

un type d'examen (FK nullable vers ExamType). Le statut (pending / confirmed / cancelled) permet de suivre l'évolution du rendez-vous.

Les FK vers ExamType et Clinic sont nullable (ON DELETE SET NULL) : si un examen ou une clinique est supprimé, le rendez-vous reste en base avec ces champs à NULL, ce qui préserve l'historique du patient.

DOCUMENT (appointments_document)

Représente un document médical déposé par le patient (ordonnance, certificat, résultats). La relation avec Appointment est une FK avec ON DELETE CASCADE : si un rendez-vous est supprimé, ses documents le sont aussi. Le fichier est stocké dans le dossier media/documents/ côté serveur.

2.2 Relations et cardinalités

Relation	De	Vers	Cardinalité & justification
est un (OneToOne)	User	Specialist	1,1 — 0,1 : un user peut être spécialiste ou simple patient
prend (patient)	User	Appointment	1,1 — 0,n : un patient peut avoir plusieurs RDV
spécialisé en	Specialty	Specialist	1,1 — 0,n : plusieurs médecins par spécialité
regroupe	Specialty	ExamType	1,1 — 0,n : plusieurs examens par spécialité
assure	Specialist	Appointment	1,1 — 0,n : un spécialiste assure plusieurs RDV
exerce dans (M2M)	Specialist	Clinic	0,n — 0,n : multi-cliniques pour un spécialiste
concerne	ExamType	Appointment	0,1 — 0,n : examen optionnel sur un RDV
se déroule dans	Clinic	Appointment	0,1 — 0,n : clinique optionnelle sur un RDV
contient	Appointment	Document	1,1 — 0,n : un RDV peut avoir plusieurs documents

3. Diagramme de Classes

Le diagramme de classes traduit le MCD en code orienté objet. Il représente les classes Python/Django avec leurs attributs typés, leurs méthodes et leurs relations d'héritage ou d'association.

Note : Le diagramme SVG du diagramme de classes est disponible en fichier joint (healthnorth_classes.svg).

3.1 Héritage Django

La classe `User` hérite de `AbstractUser` (Django built-in). `AbstractUser` fournit déjà : `username`, `password` (haché), `email`, `first_name`, `last_name`, `is_staff`, `is_active`, `date_joined`, et toute la logique d'authentification. Health North y ajoute trois attributs métier : `phone`, `address`, `date_of_birth`.

Ce choix d'héritage (plutôt que de créer un modèle `Profile` séparé) simplifie les requêtes et évite les jointures supplémentaires pour accéder aux informations de base d'un utilisateur.

3.2 Formulaires associés

Quatre formulaires Django sont rattachés au modèle `User` :

- `RegisterForm` (hérite de `UserCreationForm`) — inscription avec validation des mots de passe
- `ProfileForm` (hérite de `UserChangeForm`) — modification du profil, champ password masqué
- `PasswordChangeForm` (Django built-in) — modification sécurisée du mot de passe
- `AuthenticationForm` (Django built-in) — connexion avec vérification username/password

3.3 Méthodes `__str__`

Chaque classe implémente une méthode `__str__()` qui retourne une représentation lisible de l'objet. Ces méthodes sont utilisées automatiquement par l'interface d'administration Django et dans les templates HTML.

Classe	Retour de <code>__str__()</code>
<code>User</code>	<code>'{first_name} {last_name}'</code>
<code>Specialty</code>	<code>'{name}'</code>
<code>ExamType</code>	<code>'{name} ({duration_minutes} min)'</code>
<code>Clinic</code>	<code>'{name} — {city}'</code>
<code>Specialist</code>	<code>'Dr. {user.last_name}'</code>
<code>Appointment</code>	<code>'{patient} - {specialist} - {date}'</code>
<code>Document</code>	<code>'{type_display} — {appointment}'</code>

4. Script SQL de création de la base de données

Le script SQL ci-dessous a été généré manuellement à partir des modèles Django. Il est compatible PostgreSQL (SGBD utilisé en production via Supabase). Il inclut la création des tables, les contraintes d'intégrité, les index de performance, et des données de test.

4.1 Ordre de création des tables

Les tables sont créées dans l'ordre des dépendances pour respecter les contraintes de clés étrangères :

- 1. accounts_user (aucune dépendance)
- 2. appointments_specialty (aucune dépendance)
- 3. appointments_examtype (dépend de specialty)
- 4. appointments_clinic (aucune dépendance)
- 5. appointments_specialist (dépend de user et specialty)
- 6. appointments_specialist_clinics (dépend de specialist et clinic)
- 7. appointments_appointment (dépend de user, specialist, examtype, clinic)
- 8. appointments_document (dépend de appointment)

4.2 Choix techniques

- BIGSERIAL : clé primaire auto-incrémentée sur 64 bits (recommandé pour les grandes tables)
- TIMESTAMP WITH TIME ZONE : stockage des dates avec fuseau horaire (Django timezone-aware)
- ON DELETE CASCADE : suppression en cascade pour les documents liés à un RDV
- ON DELETE SET NULL : préservation des RDV si un examen ou une clinique est supprimé
- CONSTRAINT CHECK : validation côté base pour le statut du RDV et le type de document
- INDEX sur region et city : optimisation des requêtes API JSON du formulaire de RDV

4.3 Script complet

```
-- =====
-- HEALTH NORTH - Script de création de la base de données
-- Généré depuis les modèles Django (accounts + appointments)
-- SGBD : PostgreSQL
-- =====

-- Suppression des tables dans l'ordre inverse des dépendances
DROP TABLE IF EXISTS appointments_document CASCADE;
DROP TABLE IF EXISTS appointments_appointment CASCADE;
DROP TABLE IF EXISTS appointments_specialist_clinics CASCADE;
DROP TABLE IF EXISTS appointments_specialist CASCADE;
DROP TABLE IF EXISTS appointments_clinic CASCADE;
```

```

DROP TABLE IF EXISTS appointments_examtype CASCADE;
DROP TABLE IF EXISTS appointments_specialty CASCADE;
DROP TABLE IF EXISTS accounts_user_groups CASCADE;
DROP TABLE IF EXISTS accounts_user_user_permissions CASCADE;
DROP TABLE IF EXISTS accounts_user CASCADE;

-- =====
-- TABLE : accounts_user
-- Utilisateur étendu (hérite d'AbstractUser Django)
-- Joue le rôle de patient ET de spécialiste selon les cas
-- =====
CREATE TABLE accounts_user (
    id                BIGSERIAL PRIMARY KEY,
    password           VARCHAR(128)    NOT NULL,
    last_login         TIMESTAMP WITH TIME ZONE,
    is_superuser       BOOLEAN          NOT NULL DEFAULT FALSE,
    username           VARCHAR(150)     NOT NULL UNIQUE,
    first_name         VARCHAR(150)     NOT NULL DEFAULT '',
    last_name          VARCHAR(150)     NOT NULL DEFAULT '',
    email              VARCHAR(254)     NOT NULL DEFAULT '',
    is_staff           BOOLEAN          NOT NULL DEFAULT FALSE,
    is_active          BOOLEAN          NOT NULL DEFAULT TRUE,
    date_joined        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    -- Champs ajoutés par le projet Health North
    phone              VARCHAR(20)      NOT NULL DEFAULT '',
    address            TEXT              NOT NULL DEFAULT '',
    date_of_birth       DATE
);

-- =====
-- TABLE : accounts_user_groups (relation M2M Django auth)
-- =====
CREATE TABLE accounts_user_groups (
    id                BIGSERIAL PRIMARY KEY,
    user_id           BIGINT NOT NULL REFERENCES accounts_user(id) ON DELETE CASCADE,
    group_id          INTEGER NOT NULL
);

-- =====
-- TABLE : accounts_user_user_permissions (relation M2M Django auth)
-- =====
CREATE TABLE accounts_user_user_permissions (
    id                BIGSERIAL PRIMARY KEY,
    user_id           BIGINT NOT NULL REFERENCES accounts_user(id) ON DELETE CASCADE,
    permission_id     INTEGER NOT NULL
);

-- =====

```

```

-- TABLE : appointments_specialty
-- Spécialité médicale (ex : Cardiologie, Radiologie)
-- =====
CREATE TABLE appointments_specialty (
    id                BIGSERIAL PRIMARY KEY,
    name              VARCHAR(100) NOT NULL,
    duration_minutes  INTEGER      NOT NULL DEFAULT 30
);

-- =====
-- TABLE : appointments_examtype
-- Type d'examen médical (ex : Prise de sang, IRM, Scanner)
-- Rattaché à une spécialité via clé étrangère
-- =====
CREATE TABLE appointments_examtype (
    id                BIGSERIAL PRIMARY KEY,
    name              VARCHAR(200) NOT NULL,
    duration_minutes  INTEGER      NOT NULL DEFAULT 30,
    description        TEXT        NOT NULL DEFAULT '',
    specialty_id      BIGINT        REFERENCES appointments_specialty(id) ON
DELETE SET NULL
);

-- =====
-- TABLE : appointments_clinic
-- Clinique ou laboratoire Health North sur le territoire français
-- Région stockée sous forme de slug (ex : 'ile-de-france')
-- =====
CREATE TABLE appointments_clinic (
    id                BIGSERIAL PRIMARY KEY,
    name              VARCHAR(200) NOT NULL,
    region            VARCHAR(100) NOT NULL,
    department        VARCHAR(100) NOT NULL,
    city              VARCHAR(100) NOT NULL,
    address           VARCHAR(255) NOT NULL,
    phone             VARCHAR(20)  NOT NULL DEFAULT ''
);

-- Index pour optimiser les filtres par région et ville (utilisés dans les
API JSON)
CREATE INDEX idx_clinic_region ON appointments_clinic(region);
CREATE INDEX idx_clinic_city  ON appointments_clinic(city);

-- =====
-- TABLE : appointments_specialist
-- Médecin spécialiste lié à un compte utilisateur (OneToOne)
-- =====
CREATE TABLE appointments_specialist (
    id                BIGSERIAL PRIMARY KEY,

```



```

        bio            TEXT            NOT NULL DEFAULT '',
        user_id        BIGINT          NOT NULL UNIQUE REFERENCES accounts_user(id) ON
DELETE CASCADE,
        specialty_id  BIGINT          REFERENCES appointments_specialty(id) ON DELETE
SET NULL
    );

-- =====
-- TABLE : appointments_specialist_clinics
-- Table de liaison ManyToMany entre Specialist et Clinic
-- Un spécialiste peut exercer dans plusieurs cliniques
-- Une clinique peut accueillir plusieurs spécialistes
-- =====
CREATE TABLE appointments_specialist_clinics (
    id                BIGSERIAL PRIMARY KEY,
    specialist_id     BIGINT NOT NULL REFERENCES appointments_specialist(id) ON
DELETE CASCADE,
    clinic_id        BIGINT NOT NULL REFERENCES appointments_clinic(id)      ON
DELETE CASCADE,
    CONSTRAINT uq_specialist_clinic UNIQUE (specialist_id, clinic_id)
);

-- =====
-- TABLE : appointments_appointment
-- Rendez-vous médical entre un patient et un spécialiste
-- Statuts possibles : pending / confirmed / cancelled
-- =====
CREATE TABLE appointments_appointment (
    id                BIGSERIAL PRIMARY KEY,
    date              TIMESTAMP WITH TIME ZONE NOT NULL,
    status            VARCHAR(20) NOT NULL DEFAULT 'pending',
    notes             TEXT        NOT NULL DEFAULT '',
    patient_id        BIGINT      NOT NULL REFERENCES accounts_user(id)
ON DELETE CASCADE,
    specialist_id     BIGINT      NOT NULL REFERENCES
appointments_specialist(id) ON DELETE CASCADE,
    exam_type_id      BIGINT      REFERENCES appointments_examtype(id)
ON DELETE SET NULL,
    clinic_id         BIGINT      REFERENCES appointments_clinic(id)
ON DELETE SET NULL,
    -- Contrainte sur les valeurs du statut
    CONSTRAINT chk_status CHECK (status IN ('pending', 'confirmed',
'cancelled'))
);

-- Index pour optimiser la liste des RDV d'un patient (vue appointment_list)
CREATE INDEX idx_appointment_patient ON
appointments_appointment(patient_id);

-- =====
-- TABLE : appointments_document
-- Document médical déposé par le patient (ordonnance, certificat, etc.)

```

```

-- Stocké dans media/documents/ côté serveur
-- =====
CREATE TABLE appointments_document (
    id          BIGSERIAL      PRIMARY KEY,
    file        VARCHAR(255) NOT NULL,
    document_type VARCHAR(50) NOT NULL DEFAULT 'autre',
    uploaded_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    appointment_id BIGINT      NOT NULL REFERENCES
appointments_appointment(id) ON DELETE CASCADE,
    -- Contrainte sur les types de documents acceptés
    CONSTRAINT chk_document_type CHECK (
        document_type IN ('ordonnance', 'certificat', 'analyse', 'autre')
    )
);

-- =====
-- DONNÉES DE TEST (valeurs minimales pour validation)
-- =====

-- Spécialités
INSERT INTO appointments_specialty (name, duration_minutes) VALUES
    ('Cardiologie', 45),
    ('Radiologie', 60),
    ('Dermatologie', 30),
    ('Pédiatrie', 30),
    ('Ophtalmologie', 30);

-- Types d'examens
INSERT INTO appointments_examtype (name, duration_minutes, description,
specialty_id) VALUES
    ('Électrocardiogramme (ECG)', 30, 'Enregistrement de l''activité
électrique du cœur', 1),
    ('Échographie cardiaque', 60, 'Imagerie du cœur par ultrasons',
1),
    ('IRM cérébrale', 60, 'Imagerie par résonance magnétique
du cerveau', 2),
    ('Scanner thoracique', 45, 'Tomodensitométrie de la zone
thoracique', 2),
    ('Dermoscopie', 20, 'Examen de la peau par dermoscope',
3),
    ('Biopsie cutanée', 30, 'Prélèvement d''un échantillon de
peau', 3),
    ('Consultation pédiatrique', 30, 'Consultation générale pour
enfants', 4),
    ('Bilan de croissance', 45, 'Suivi de la croissance de
l''enfant', 4),
    ('Fond d''œil', 20, 'Examen de la rétine',
5),
    ('Champ visuel', 30, 'Test du champ de vision',
5),
    ('Prise de sang', 15, 'Prélèvement sanguin pour analyse',
NULL),

```

```

        ('Radiographie',                20, 'Imagerie par rayons X',
2),
        ('Échographie abdominale',      45, 'Imagerie de l''abdomen par
ultrasons',                2),
        ('Consultation générale',        30, 'Consultation médicale générale',
NULL);

-- Cliniques
INSERT INTO appointments_clinic (name, region, department, city, address,
phone) VALUES
    ('Clinique Saint-Louis',            'ile-de-france',        'Paris',
'Paris',          '12 rue de la Paix, 75001 Paris',          '01 42 00 00 01'),
    ('Centre Médical Lyon Sud',          'auvergne-rhone-alpes', 'Rhône',
'Lyon',           '5 avenue Jean Jaurès, 69007 Lyon',          '04 72 00 00 02'),
    ('Clinique Méditerranée',            'provence-alpes-cote-azur', 'Bouches-du-
Rhône', 'Marseille', '8 boulevard Michelet, 13008 Marseille',          '04 91 00
00 03'),
    ('Centre de Santé Bordeaux',          'nouvelle-aquitaine',    'Gironde',
'Bordeaux', '3 cours du Chapeau Rouge, 33000 Bordeaux',          '05 56 00 00 04'),
    ('Clinique du Nord',                  'hauts-de-france',        'Nord',
'Lille',          '15 rue Nationale, 59000 Lille',          '03 20 00 00 05'),
    ('Centre Médical Toulouse',            'occitanie',              'Haute-
Garonne', 'Toulouse', '22 allées Jean Jaurès, 31000 Toulouse',          '05 61
00 00 06');

```

5. Architecture technique de déploiement

L'application est déployée en production avec l'architecture suivante :

Composant	Technologie	Rôle
Serveur d'application	Render.com (Free tier)	Héberge l'application Django via Unicorn
Base de données	Supabase (PostgreSQL)	Stockage persistant des données
Serveur WSGI	Gunicorn	Interface entre Render et Django
Fichiers statiques	WhiteNoise	Sert CSS/JS directement depuis Django
Versionning	Git + GitHub	Code source et déploiement continu
Framework web	Django 6 / Python 3.12	Backend MVC
Frontend	Bootstrap 5 + JavaScript	Interface responsive

5.1 Variables d'environnement

Les paramètres sensibles sont externalisés dans des variables d'environnement Render.com, conformément aux bonnes pratiques de sécurité (pas de secrets dans le code source) :

- SECRET_KEY — clé secrète Django
- DEBUG — False en production
- DATABASE_URL — URL de connexion à Supabase PostgreSQL
- PYTHON_VERSION — version Python imposée à Render

5.2 Processus de déploiement

À chaque push sur la branche main, Render exécute automatiquement le script build.sh qui :

- Installe les dépendances (pip install -r requirements.txt)
- Collecte les fichiers statiques (python manage.py collectstatic)
- Applique les migrations (python manage.py migrate)
- Charge les données de référence (python manage.py loaddata)
- Crée les comptes de test (python manage.py seed)

6. Aspects sécurité

Plusieurs mesures de sécurité ont été implémentées en réponse aux risques identifiés dans l'analyse ISO 27001 du cahier des charges :

Risque identifié	Mesure implémentée
Accès non authentifié	Décorateur @login_required sur toutes les vues sensibles
Accès aux données d'autrui	Filtrage patient=request.user sur tous les RDV et documents
Mots de passe faibles	Django impose PBKDF2-SHA256 + validateurs de complexité
CSRF	Protection Django CSRF activée par défaut sur tous les formulaires
Secrets exposés	SECRET_KEY et DATABASE_URL en variables d'environnement
Upload malveillant	Stockage dans media/documents/ hors racine web publique

7. Conclusion

La base de données Health North est structurée autour de 8 tables couvrant l'ensemble du parcours patient : de la création du compte à la prise de rendez-vous et au dépôt de documents. Les choix de modélisation (héritage AbstractUser, OneToOne pour Specialist, ManyToMany pour les cliniques, FK nullable pour préserver l'historique) ont été guidés par les besoins métier et les bonnes pratiques Django.

L'application est déployée en production sur <https://healthnorth.onrender.com> et accessible avec les comptes de test fournis dans le dossier.